

Relatório Técnico e Teórico das Modificações do Projeto PESCD

Análise de Evolução Arquitetural, Engenharia de Requisitos e Padrões RESTful / Securitários

1. Visão Geral

O projeto **PESCD** é um sistema de software corporativo-acadêmico projetado para o gerenciamento de ponta a ponta do *Programa de Estágio Supervisionado de Capacitação Docente*. Desenvolvida em Java com o ecossistema Spring Boot, a aplicação adota uma arquitetura em camadas bem definida, garantindo modularidade e isolamento de escopo técnico através de controllers, services, repositories, models, DTOs, componentes de segurança perimetral e interfaces Thymeleaf para entrega legada.

As modificações arquiteturais consolidadas realizam a transição estrutural de um monolito tradicional baseado em acoplamento severo de telas por MVC (Server-Side Rendering) para um ecossistema híbrido moderno que expõe sub-recursos através de uma API puramente RESTful, protegida por persistência relacional transacional e controle de acessos federados via tokens assíncronos JWT.

Com base na análise de rastreabilidade de engenharia de software do projeto, destacam-se três marcos fundamentais:

- **PESCD + Mysql + JWT:** Engenharia da infraestrutura base da aplicação, parametrização da camada de persistência com isolamento transacional, implementação do núcleo do Spring Security e controle de acessos RBAC.
- **Aluno e Documentos REST:** Exposição desacoplada das funcionalidades relativas à jornada acadêmica do discente, permitindo submissão de payloads complexos multipartes e verificação assíncrona de status de documentos.
- **Admin REST:** Conversão completa do gerenciamento administrativo de usuários do padrão relacional monolítico para endpoints REST limpos de CRUD.

2. Tecnologias e Fundamentos Utilizados

A aplicação foi atualizada e opera sobre o **Spring Boot 3.5.14** em consonância com as otimizações do **Java 21** (como os avanços de performance em concorrência, coleções e correspondência de padrões). O uso do Spring Boot mitiga o esforço de configurações via XML ou heranças boilerplate por meio de starters autoconfiguráveis estruturados via Apache Maven:

Dependência Starter (Maven)	Finalidade e Justificativa de Engenharia
spring-boot-starter-web	Gerenciamento do servidor embutido Tomcat e mapeamento do DispatcherServlet para suporte a endpoints REST com Jackson Serialization.
spring-boot-starter-data-jpa	Abstração de acesso a dados com suporte nativo ao Hibernate Core, critérios de consulta estáticos e controle declarativo de transações.
spring-boot-starter-security	

Dependência Starter (Maven)	Finalidade e Justificativa de Engenharia
	Segurança perimetral, barramento de filtros HTTP encadeados (FilterChain) e proteção ativa contra vulnerabilidades web.
<code>spring-boot-starter-validation</code>	Aplicação declarativa de restrições através do Hibernate Validator de acordo com a especificação Jakarta Validation.
<code>io.jsonwebtoken (jjwt)</code>	Biblioteca para codificação, decodificação, criptografia simétrica e verificação de assinatura digital de tokens estruturados JWT.

3. Arquitetura em Camadas e Separação de Conceitos

O código-fonte segue estritamente a separação de conceitos (*Separation of Concerns - SoC*), estruturando as responsabilidades de forma que a integridade arquitetural do sistema seja mantida imune ao vazamento de escopo:

A) Controllers (Adaptadores de Entrada): Camada de interface de rede responsável exclusiva pelo mapeamento de rotas e verbos HTTP, extração de parâmetros de URL ou corpos de mensagens (JSON), validações estruturais de integridade e delegação imediata para as camadas subsequentes.

B) Services (Camada de Domínio / Negócio): Onde reside o coração da aplicação. Centraliza as validações lógicas e transições controladas da máquina de estados do negócio (ex: fluxos de deferimento e validação de PDFs), realizando a orquestração segura das transações com o banco através da anotação `@Transactional`.

C) Repositories (Persistência): Interfaces limpas orientadas ao Spring Data JPA que eliminam a necessidade de escrita manual de códigos SQL ou gerenciamento direto de conexões, provendo consultas seguras e tipadas baseadas em convenção de nomenclatura ou JPQL.

4. Persistência e Modelo de Domínio Relacional

O mapeamento objeto-relacional (ORM) vincula com precisão o modelo de dados em MySQL com a lógica orientada a objetos das entidades do sistema. O relacionamento centraliza-se na entidade associativa `AlunoOferta`, que atua como o agregado principal controlador da evolução do estágio de um aluno em uma respectiva disciplina ofertada.

Padrão de Agregação de Negócio: Artefatos como `PlanoTrabalho`, `DocumentacaoDocencia` e `RelatorioFinal` encontram-se vinculados diretamente à chave estrangeira da matrícula do aluno (`AlunoOferta`), assegurando que o histórico de transições de status seja preservado de forma totalmente auditável.

5. Segurança Perimetral, Autenticação e Autorizacao

A arquitetura de segurança foi reconfigurada para estabelecer uma política **totalmente stateless (sem estado)** na classe `SecurityConfig`. Foram removidos os acoplamentos legados de sessão por cookies

(JSESSIONID), formulários nativos de login e mecanismos básicos de autenticação HTTP. Esse modelo é obrigatório para APIs REST escaláveis.

O fluxo base mapeado em [/api/auth/login](#) encapsula as credenciais e as valida via `AuthenticationManager`. Se válidas, o `JwtService` gera um Bearer Token criptografado assinado com algoritmo simétrico robusto HMAC-SHA, contendo dados do usuário e suas authorities incorporadas como claims customizadas.

O controle de acessos baseia-se em perfis institucionais estruturados através de controle baseado em papéis (*Role-Based Access Control - RBAC*):

- **ROLE_ADMINISTRADOR**: Permissão total ao escopo do CRUD de usuários, sob auditoria e proteção de segurança contra autoexclusão direta.
- **ROLE_ALUNO**: Permissão de submissão de artefatos multipartes ligados às suas respectivas ofertas matriculadas.
- **ROLE_SUPERVISOR** e **ROLE_RESPONSAVEL**: Escopo voltado à avaliação, deferimento e encerramento de atividades de estágio.

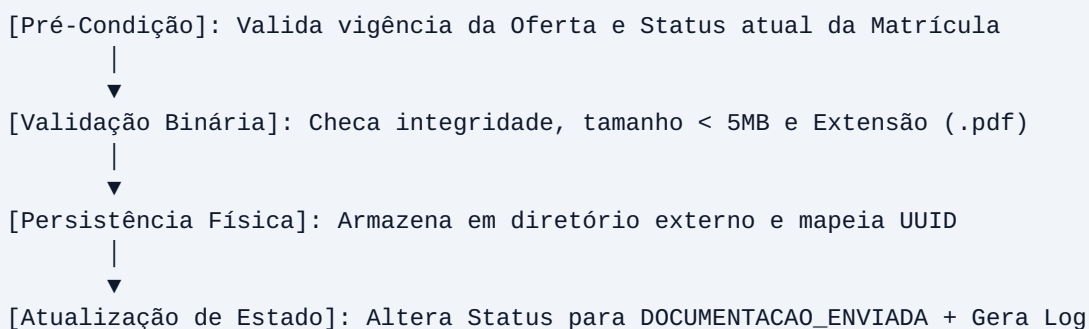
6. Módulo Discente e Submissão Multipartes via REST

A substituição do fluxo ThymeLeaf legada pela abordagem REST expôs a jornada do aluno de maneira desacoplada. O uso da anotação `@AuthenticationPrincipal` nos controladores extrai de forma automática a identidade do usuário a partir do contexto do token JWT validado, mitigando totalmente falhas graves de Broken Object Level Authorization (IDOR) — impedindo que um aluno visualize ou submeta arquivos em nome de outro usuário maliciosamente.

Para o upload de documentos de estágio, o sistema consome payloads mapeados como `multipart/form-data`. Esse formato é o padrão ideal para o transporte combinado de strings estruturadas (como metadados de identificadores) e fluxos binários complexos (arquivos PDF).

7. Máquina de Estados e Transições de Status Remotas

Os serviços associados à documentação coordenam uma máquina de estados estrita para o ciclo de vida do estágio, impedindo transições ilegais ou corrupção de fluxo lógico. O tratamento aplicado ao `DocumentacaoDocenciaService` serve de referência arquitetural:



8. Módulo Administrativo e Desacoplamento via DTOs

No commit Admin REST, a classe AdminUsuarioController foi totalmente convertida para @RestController. Eliminou-se por completo a poluição de código gerada pelo controle de atributos de tela via instâncias de Model e redirecionamentos por strings de caminhos físicos de arquivos HTML.

A comunicação opera via consumo e emissão estrita de payloads JSON associados ao padrão **DTO (Data Transfer Object)** através da classe AdminUsuarioForm. A utilização de DTOs isola as entidades JPA de persistência do contrato público exposto à rede, mitigando vulnerabilidades críticas de *Mass Assignment* (onde parâmetros maliciosos poderiam injetar dados diretamente em colunas protegidas do banco).

9. Engenharia de Resolução de Bugs e Casos de Sucesso

Durante os ciclos de testes integrados e validações das modificações REST, três problemas críticos de engenharia foram mitigados com sucesso:

A) Resiliência e Status Semântico de Consultas Vazias (GET): Anteriormente, se um aluno realizava uma consulta GET a um documento ainda não submetido, o sistema lançava uma exceção de infraestrutura estourando um erro 404 Not Found. Corrigiu-se a lógica de negócio para que o Service identifique a validade da matrícula e, caso o arquivo esteja ausente, retorne um status 200 OK acompanhado do DTO com o campo estático contendo o metadado `NAO_ENVIADO` e as demais propriedades nulas, fornecendo os subsídios necessários para o front-end renderizar os componentes de tela adequados.

B) Resolução de Incompatibilidade de UUID Binário: O Hibernate JPA mapeia chaves nativas do tipo UUID como arranjos compactos de 16 bytes no MySQL (BINARY(16)), gerando caracteres de texto corrompidos quando visualizados incorretamente. Ajustou-se a desserialização do Jackson nos DTOs para forçar a validação estrita baseada na especificação **RFC-4122** com a inclusão obrigatória de hifens separadores, eliminando o erro `MethodArgumentTypeMismatchException`.

C) Resolução do Erro Multipart Boundary: Corrigiu-se o erro `MultipartException: no multipart boundary was found` eliminando a declaração explícita manual do cabeçalho Content-Type no cliente HTTP (Postman). Isso permitiu que o motor de rede calculasse de forma nativa a string de fronteira (boundary) essencial para separar os blocos textuais do arquivo binário anexado.

10. Qualidade de Software, Riscos e Próximos Passos

Embora as modificações confirmem um alto nível de maturidade técnica ao ecossistema do projeto PESCD, mapeou-se a necessidade de melhorias contínuas para mitigar riscos de regressão em ambientes de produção real:

- **Ampliação de Testes Automatizados:** Desenvolvimento urgente de baterias de testes unitários e de integração mockando o Spring Security (@WithMockUser) para validar o comportamento dos endpoints de upload de arquivos e transições de status da máquina de estados sob estresse.
- **Reforço na Segurança de Arquivos:** Implementar a leitura ativa de *magic bytes* (MIME-type real do arquivo) na camada de serviço para garantir a exclusividade de submissão de PDFs legítimos, contornando tentativas de burlar o sistema alterando apenas a extensão de arquivos executáveis potencialmente nocivos.